

SETTINGS AND PERMISSIONS

Configure permissions

Control what Claude Code can access and do with fine-grained permission rules, modes, and managed policies.

Claude Code supports fine-grained permissions so that you can specify exactly what the agent is allowed to do and what it can't. Permission settings can be checked into version control and distributed to all developers in your organization, as well as customized by individual developers.

Permission system

Claude Code uses a tiered permission system to balance power and safety:

Tool type	Example	Approval required	"Yes, don't ask again" behavior
Read-only	File reads, Grep	No	N/A
Bash commands	Shell execution	Yes	Permanently per project directory and command
File modification	Edit/write files	Yes	Until session end

Manage permissions

You can view and manage Claude Code's tool permissions with `/permissions`. This UI lists all permission rules and the `settings.json` file each rule comes from.

- **Allow** rules let Claude Code use the specified tool without manual approval.
- **Ask** rules prompt for confirmation whenever Claude Code tries to use the specified tool.
- **Deny** rules prevent Claude Code from using the specified tool.

Rules are evaluated in order: deny, then ask, then allow. The first match in that order determines the outcome, and rule specificity doesn't change the order.

A broad deny rule like `Bash(aws *)` blocks every matching call, including calls that also match a narrower allow rule like `Bash(aws s3 ls)`, so a deny rule can't carry allowlist exceptions. The same precedence applies between ask and allow: a matching ask rule prompts even when a more specific allow rule also matches the same call.

Deny rules behave differently depending on whether they name a tool or scope a pattern within one. A bare tool name like `Bash` removes the tool from Claude's context entirely, so Claude never sees it. A scoped rule like `Bash(rm *)` leaves the tool available and blocks matching calls when Claude attempts them.

 Permission rules are enforced by Claude Code, not by the model. Instructions in your prompt or `CLAUDE.md` shape what Claude tries to do, but they don't change what Claude Code allows. To grant or revoke access, use `/permissions`, the rules described here, a [permission mode](#), or a [PreToolUse hook](#).

Permission modes

Claude Code supports several permission modes that control how it approves tool calls. See [Permission modes](#) for when to use each one. Set the `defaultMode` in your [settings files](#):

Mode	Description
<code>default</code>	Standard behavior: prompts for permission on first use of each tool
<code>acceptEdits</code>	Automatically accepts file edits and common filesystem commands such as <code>mkdir</code> , <code>touch</code> , <code>mv</code> , and <code>cp</code> for paths in the working directory or <code>additionalDirectories</code>
<code>plan</code>	Plan Mode: Claude reads files and runs read-only shell commands to explore but doesn't edit your source files
<code>auto</code>	Auto-approves tool calls with background safety checks that verify actions align with your request. Currently a research preview
<code>dontAsk</code>	Auto-denies tools unless pre-approved via <code>/permissions</code> or <code>permissions.allow</code> rules
<code>bypassPermissions</code>	Skips permission prompts, except those forced by explicit <code>ask</code> rules. Root and home directory removals such as <code>rm -rf /</code> also still prompt as a circuit breaker

 `bypassPermissions` mode skips permission prompts, including for writes to `.git`, `.config/git`, `.claude`, `.vscode`, `.idea`, `.husky`, `.cargo`, `.devcontainer`, `.yarn`, and `.mvn`. Explicit `ask` rules still force a prompt, and removals targeting the filesystem root or home directory, such as `rm -rf /` and `rm -rf ~`, still prompt as a circuit breaker against model error. Only use this mode in isolated environments like containers or VMs where Claude Code can't cause damage.

To prevent `bypassPermissions` or `auto` mode from being used, set `permissions.disableBypassPermissionsMode` or `permissions.disableAutoMode` to "disable" in any settings file. These are most useful in managed settings where they can't be overridden.

Permission rule syntax

Permission rules follow the format `Tool` or `Tool(specifier)` .

Match all uses of a tool

To match all uses of a tool, use only the tool name without parentheses:

Rule	Effect
<code>Bash</code>	Matches all Bash commands
<code>WebFetch</code>	Matches all web fetch requests
<code>Read</code>	Matches all file reads

`Bash(*)` is equivalent to `Bash` and matches all Bash commands. As a deny rule, both forms remove the tool from Claude's context.

Use specifiers for fine-grained control

Add a specifier in parentheses to match specific tool uses:

Rule	Effect
<code>Bash(npm run build)</code>	Matches the exact command <code>npm run build</code>
<code>Read(./ .env)</code>	Matches reading the <code>.env</code> file in the current directory
<code>WebFetch(domain:example.com)</code>	Matches fetch requests to example.com

Match by input parameter

Deny and ask rules can match a top-level input parameter on any tool with `Tool(param:value)` . The rule matches when Claude calls the tool with that parameter set to that exact value. An allow rule for one parameter value wouldn't establish that the call is safe overall, so allow rules continue to use

each tool’s own specifier syntax. This works for any scalar parameter the tool accepts:

Rule	Matches
Agent(model:opus)	Agent calls that request the Opus model tier
Agent(isolation:worktree)	Agent calls that request a git worktree
Bash(run_in_background:true)	Bash calls that run in the background

Parameter matching follows these rules:

- The parameter name must be a direct field of the tool’s input, such as `model` on the Agent tool. Fields nested inside an object or array are not matchable
- Each rule names one parameter. To gate on both `model` and `isolation` , write two rules, `Agent(model:opus)` and `Agent(isolation:worktree)` , rather than combining them in one rule
- The value supports `*` as a wildcard that matches any sequence of characters, so `Agent(isolation:*)` matches any explicit isolation value. Without `*` the match is exact
- A parameter the model omits is never matched, so `Agent(model:*)` doesn’t match a call that leaves `model` unset
- The value is compared against the literal input Claude sends, before any normalization. `Agent(model:opus)` matches the alias `opus` but not a full model ID. Run with `--verbose` to see the exact parameter names and values in each tool call
- Whitespace around the colon is ignored

Fields that a tool already matches with its own canonicalizing rules are not matchable this way:

`command` for Bash and PowerShell, `file_path` for Read, Edit, and Write, `path` for Grep and Glob, `notebook_path` for NotebookEdit, and `url` for WebFetch. A rule like `Bash(command:rm *)` would be bypassable by a compound command, so Claude Code ignores it and emits a startup warning. Use `Bash(rm *)` , `Read(./path)` , or `WebFetch(domain:host)` instead.

Wildcard patterns

Bash rules support glob patterns with `*` . Wildcards can appear at any position in the command. This configuration allows npm and git commit commands while blocking git push:

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git commit *)",
      "Bash(git * main)",
      "Bash(* --version)",
      "Bash(* --help *)"
    ],
    "deny": [
      "Bash(git push *)"
    ]
  }
}
```

The space before `*` matters: `Bash(ls *)` matches `ls -la` but not `ls of`, while `Bash(ls*)` matches both. The `:*` suffix is an equivalent way to write a trailing wildcard, so `Bash(ls:*)` matches the same commands as `Bash(ls *)`.

The permission dialog writes the space-separated form when you select “Yes, don’t ask again” for a command prefix. The `:*` form is only recognized at the end of a pattern. In a pattern like `Bash(git:* push)`, the colon is treated as a literal character and won’t match git commands.

Tool name wildcards

Deny and ask rules also accept glob patterns in the tool-name position. The pattern must match the full tool name: `"*"` matches every tool, and `"mcp__*"` matches every MCP tool across all servers. A tool matched by a bare-name glob deny rule is removed from Claude’s context, the same as a bare tool name. This configuration denies every MCP tool:

```
{
  "permissions": {
    "deny": [
      "mcp__*"
    ]
  }
}
```

Allow rules accept tool-name globs only after a literal `mcp__<server>__` prefix. The server segment

must be glob-free so the rule names a specific server you configured. `mcp__puppeteer__*` matches every tool from the `puppeteer` server, and `mcp__github__get_*` matches its `get_` tools. An unanchored allow glob such as `"*"`, `"B*"`, or `"mcp__*"` is skipped with a warning and doesn't auto-approve anything.

A deny or ask rule whose tool name matches no known tool produces a startup warning to catch typos. Tool names containing `_` or `*` are exempt from the check.

The label shown for a tool in the transcript and permission dialog can differ from its canonical name. For example, the tool labeled `Stop Task` in the transcript has the canonical name `TaskStop`. Permission rules and **hook matchers** match the canonical name only, so a rule written as `Stop Task` doesn't match. For deny and ask rules, the startup warning above catches the mismatch. Use the canonical names listed in the **tools reference**.

Tool-specific permission rules

Bash

Bash permission rules support wildcard matching with `*`. Wildcards can appear at any position in the command, including at the beginning, middle, or end:

- `Bash(npm run build)` matches the exact Bash command `npm run build`
- `Bash(npm run test *)` matches Bash commands starting with `npm run test`
- `Bash(npm *)` matches any command starting with `npm`
- `Bash(* install)` matches any command ending with `install`
- `Bash(git * main)` matches commands like `git checkout main` and `git log --oneline main`

A single `*` matches any sequence of characters including spaces, so one wildcard can span multiple arguments. `Bash(git *)` matches `git log --oneline --all`, and `Bash(git * main)` matches `git push origin main` as well as `git merge main`.

When `*` appears at the end with a space before it (like `Bash(ls *)`), it enforces a word boundary, requiring the prefix to be followed by a space or end-of-string. For example, `Bash(ls *)` matches `ls -la` but not `ls of`. In contrast, `Bash(ls*)` without a space matches both `ls -la` and `ls of` because there's no word boundary constraint.

Compound commands

💡 Claude Code is aware of shell operators, so a rule like `Bash(safe-cmd *)` won't give it permission to run the command `safe-cmd && other-cmd`. The recognized command separators are `&&`, `||`, `;`, `|`, `|&`, `&`, and newlines. A rule must match each subcommand independently.

When you approve a compound command with “Yes, don't ask again”, Claude Code saves a separate rule for each subcommand that requires approval, rather than a single rule for the full compound string. For example, approving `git status && npm test` saves a rule for `npm test`, so future `npm test` invocations are recognized regardless of what precedes the `&&`. Subcommands like `cd` into a subdirectory generate their own Read rule for that path. Up to 5 rules may be saved for a single compound command.

Process wrappers

Before matching Bash rules, Claude Code strips a fixed set of process wrappers so a rule like `Bash(npm test *)` also matches `timeout 30 npm test`. The recognized wrappers are `timeout`, `time`, `nice`, `nohup`, and `stdbuf`.

Bare `xargs` is also stripped, so `Bash(grep *)` matches `xargs grep pattern`. Stripping applies only when `xargs` has no flags: an invocation like `xargs -n1 grep pattern` is matched as an `xargs` command, so rules written for the inner command do not cover it.

This wrapper list is built in and is not configurable. Development environment runners such as `direnv exec`, `devbox run`, `mise exec`, `npx`, and `docker exec` are not in the list. Because these tools execute their arguments as a command, a rule like `Bash(devbox run *)` matches whatever comes after `run`, including `devbox run rm -rf .`. To approve work inside an environment runner, write a specific rule that includes both the runner and the inner command, such as `Bash(devbox run npm test)`. Add one rule per inner command you want to allow.

Exec wrappers such as `watch`, `setsid`, `ionice`, and `flock` always prompt and can't be auto-approved by a prefix rule like `Bash(watch *)`. The same applies to `find` with `-exec` or `-delete`: a `Bash(find *)` rule doesn't cover these forms. To approve a specific invocation, write an exact-match rule for the full command string.

Read-only commands

Claude Code recognizes a built-in set of Bash commands as read-only and runs them without a permission prompt in every mode. These include `ls`, `cat`, `echo`, `pwd`, `head`, `tail`, `grep`, `find`, `wc`, `which`, `diff`, `stat`, `du`, `cd`, and read-only forms of `git`. The set is not

configurable; to require a prompt for one of these commands, add an `ask` or `deny` rule for it.

Unquoted glob patterns are permitted for commands whose every flag is read-only, so `ls *.ts` and `wc -l src/*.py` run without a prompt. Commands with write-capable or exec-capable flags, such as `find`, `sort`, `sed`, and `git`, still prompt when an unquoted glob is present because the glob could expand to a flag like `-delete`.

A `cd` into a path inside your working directory or an **additional directory** is also read-only. A compound command like `cd packages/api && ls` runs without a prompt when each part qualifies on its own. Combining `cd` with `git` in one compound command always prompts, regardless of the target directory.

⚠ Bash permission patterns that try to constrain command arguments are fragile. For example, `Bash(curl http://github.com/ *)` intends to restrict curl to GitHub URLs, but won't match variations like:

- Options before URL: `curl -X GET http://github.com/...`
- Different protocol: `curl https://github.com/...`
- Redirects: `curl -L http://bit.ly/xyz`, which redirects to GitHub
- Variables: `URL=http://github.com && curl $URL`
- Extra spaces: `curl http://github.com`

For more reliable URL filtering, consider:

- **Restrict Bash network tools:** use deny rules to block `curl`, `wget`, and similar commands, then use the WebFetch tool with `WebFetch(domain:github.com)` permission for allowed domains
- **Use PreToolUse hooks:** implement a hook that validates URLs in Bash commands and blocks disallowed domains
- **Add CLAUDE.md guidance:** describe your allowed curl patterns in `CLAUDE.md`. This shapes what Claude tries but doesn't enforce a boundary, so pair it with one of the options above

Note that using WebFetch alone doesn't prevent network access. If Bash is allowed, Claude can still use `curl`, `wget`, or other tools to reach any URL.

PowerShell

PowerShell permission rules use the same shape as Bash rules. Wildcards with `*` match at any position, the `:*` suffix is equivalent to a trailing `*`, and a bare `PowerShell` or `PowerShell(*)` matches every command. This configuration allows `Get-ChildItem` and `git commit` commands while blocking `Remove-Item`:


```
{
  "permissions": {
    "allow": [
      "PowerShell(Get-ChildItem *)",
      "PowerShell(git commit *)"
    ],
    "deny": [
      "PowerShell(Remove-Item *)"
    ]
  }
}
```

Common aliases are canonicalized before matching. A rule written for the cmdlet name also matches its aliases, so `PowerShell(Get-ChildItem *)` matches `gci`, `ls`, and `dir` as well. Matching is case-insensitive.

Claude Code parses the PowerShell AST and checks each command in a compound command independently. Pipeline operators `|`, statement separators `;`, and on PowerShell 7+ the chain operators `&&` and `||` split a compound command into subcommands. A rule must match every subcommand for the compound command to be allowed.

Read and Edit

Edit rules apply to all built-in tools that edit files. Claude makes a best-effort attempt to apply **Read** rules to all built-in tools that read files like Grep and Glob, to `@file` mentions in your prompts, and to the selection and open-file context that a connected **IDE** shares with Claude.

 Read and Edit deny rules apply to Claude's built-in file tools and to file commands Claude Code recognizes in Bash, such as `cat`, `head`, `tail`, and `sed`. They don't apply to arbitrary subprocesses that read or write files indirectly, like a Python or Node script that opens files itself. For OS-level enforcement that blocks all processes from accessing a path, **enable the sandbox**.

Read and Edit rules both follow the **gitignore** specification with four distinct pattern types:

Pattern	Meaning	Example	Matches
<code>//path</code>	Absolute path from filesystem root	<code>Read(//Users/alice/secrets/**)</code>	<code>/Users/alice/secrets/**</code>
<code>~/path</code>	Path from home directory	<code>Read(~/Documents/*.pdf)</code>	<code>/Users/alice/Documents/*.pdf</code>

Pattern	Meaning	Example	Matches
/path	Path relative to project root	Edit(/src/**/*.ts)	<project root>/src/**/*.ts
path or ./path	Path relative to current directory	Read(*.env)	<cwd>/*.env

⚠ A pattern like /Users/alice/file isn't an absolute path. It's relative to the project root. Use //Users/alice/file for absolute paths.

On Windows, paths are normalized to POSIX form before matching. C:\Users\alice becomes /c/Users/alice , so use //c/**/*.env to match .env files anywhere on that drive. To match across all drives, use /**/*.env .

Examples:

- Edit(/docs/**) : edits in <project>/docs/ , not /docs/ or <project>/.claude/docs/
- Read(~/.zshrc) : reads your home directory's .zshrc
- Edit(/tmp/scratch.txt) : edits the absolute path /tmp/scratch.txt
- Read(src/**) : reads from <current-directory>/src/

A rule only matches files under its anchor, so the anchor determines how far a deny rule reaches. Bare filenames follow gitignore semantics and match at any depth, so Read(.env) and Read(**/.env) are equivalent:

Deny rule	Blocks	Does not block
Read(.env) or Read(**/.env)	any .env at or under the current directory	.env in a parent directory or another project
Read(/**/.env)	any .env anywhere on the filesystem	nothing; the rule is anchored at the filesystem root

❗ In gitignore patterns, * matches within a single path segment and can appear at any position in the pattern, while ** matches across directories. To allow all file access, use only the tool name without parentheses: Read , Edit , or Write .

When Claude accesses a symlink, permission rules check two paths: the symlink itself and the file it resolves to. Allow and deny rules treat that pair differently: allow rules fall back to prompting you, while deny rules block outright.

- **Allow rules:** apply only when both the symlink path and its target match. A symlink inside an allowed directory that points outside it still prompts you.
- **Deny rules:** apply when either the symlink path or its target matches. A symlink that points to a denied file is itself denied.

For example, with `Read(./project/**)` allowed and `Read(~/.ssh/**)` denied, a symlink at `./project/key` pointing to `~/.ssh/id_rsa` is blocked: the target fails the allow rule and matches the deny rule.

WebFetch

WebFetch rules use a `domain:` prefix and match against the hostname of the requested URL. Matching is case-insensitive, supports `*` wildcards, and strips a trailing `.` from both the rule and the hostname so `example.com.` and `example.com` are treated the same.

- `WebFetch(domain:example.com)` matches requests to `example.com`
- `WebFetch(domain:*.example.com)` matches any subdomain at any depth, such as `api.example.com` or `a.b.example.com`, but not `example.com` itself
- `WebFetch(domain:*)` matches every domain and is equivalent to a bare `WebFetch` rule

In any position other than a leading `.*` or a bare `*`, the wildcard matches only the text between two dots. `WebFetch(domain:example.*)` matches `example.org`, where `*` becomes `org`, but not `example.evil.com`, where `*` would have to become `evil.com` and cross a dot. This keeps a trailing wildcard from matching domains an attacker could register.

MCP

MCP rules use the server name as configured in Claude Code, optionally followed by the name of a tool from that server.

- `mcp__puppeteer` matches any tool provided by the `puppeteer` server
- `mcp__puppeteer__*` uses wildcard syntax and also matches all tools from the `puppeteer` server
- `mcp__puppeteer__puppeteer_navigate` matches the `puppeteer_navigate` tool provided by the `puppeteer` server

Agent (subagents)

Use `Agent(AgentName)` rules to control which subagents Claude can use:

- `Agent(Explore)` matches the Explore subagent
- `Agent(Plan)` matches the Plan subagent
- `Agent(my-custom-agent)` matches a custom subagent named `my-custom-agent`

Add these rules to the `deny` array in your settings or use the `--disallowedTools` CLI flag to disable specific agents. To disable the Explore agent:

```
{
  "permissions": {
    "deny": ["Agent(Explore)"]
  }
}
```

Cd

`Cd` rules control which directories the /cd command can move the session to. `Cd` is not a model-invocable tool: Claude can't call it, and the rules apply only when you run `/cd` yourself.

A bare `Cd` deny rule disables `/cd` entirely. A `Cd(<path-pattern>)` deny rule blocks matching targets. Deny rules check every spelling of the target, including each symlink hop it resolves through, so a rule written for one path also blocks targets that resolve to it.

Adding any `Cd` allow rule switches `/cd` to allowlist mode: the resolved target directory must match one of your allow rules, or `/cd` refuses. With no `Cd` rules configured, `/cd` keeps its default behavior and prompts you to trust an unfamiliar directory.

Path patterns share the `//`, `~/`, and `/` anchors from Read and Edit rules, but matching is anchored to the whole directory path rather than gitignore-style. `*` matches exactly one path segment and `**` matches across segments. A trailing `/**` also matches its named root.

Rule	Matches	Does not match
<code>Cd(~ / code / *)</code>	<code>~/code/app</code>	<code>~/code/app/src</code> , <code>~/code</code>
<code>Cd(~ / code / **)</code>	<code>~/code</code> and any directory under it	directories outside <code>~/code</code>
<code>Cd(** / node_modules)</code>	any <code>node_modules</code> directory at any depth	<code>node_modules/pkg</code>

Extend permissions with hooks

Claude Code hooks provide a way to register custom shell commands to perform permission evaluation at runtime. When Claude Code makes a tool call, PreToolUse hooks run before the permission prompt. The hook output can deny the tool call, force a prompt, or skip the prompt to let the call proceed.

Hook decisions don't bypass permission rules. Deny and ask rules are evaluated regardless of what a PreToolUse hook returns, so a matching deny rule blocks the call and a matching ask rule still prompts even when the hook returned `"allow"` or `"ask"`. This preserves the deny-first precedence described in **Manage permissions**, including deny rules set in managed settings.

A blocking hook also takes precedence over allow rules. A hook that exits with code 2 stops the tool call before permission rules are evaluated, so the block applies even when an allow rule would otherwise let the call proceed. To run all Bash commands without prompts except for a few you want blocked, add `"Bash"` to your allow list and register a PreToolUse hook that rejects those specific commands. See **Block edits to protected files** for a hook script you can adapt.

Working directories

By default, Claude has access to files in the directory where you launched it. You can extend this access:

- **During startup:** use `--add-dir <path>` CLI argument
- **During session:** use `/add-dir` command
- **Persistent configuration:** add to `additionalDirectories` in **settings files**

Files in additional directories follow the same permission rules as the original working directory: they become readable without prompts, and file editing permissions follow the current permission mode.

To change the session's primary working directory instead of adding another, use `/cd`. The `/cd` command requires Claude Code v2.1.169 or later. Unlike `/add-dir`, it relocates the session: the new directory's `CLAUDE.md` is loaded and `--resume` finds the session from there.

Additional directories grant file access, not configuration

Adding a directory extends where Claude can read and edit files. It doesn't make that directory a full configuration root: most `.claude/` configuration is not discovered from additional directories, though a few types are loaded as exceptions.

These exceptions apply only to directories added with the `--add-dir` flag or the `/add-dir` command. Directories listed in `permissions.additionalDirectories` in a settings file grant file access only and don't load any of the configuration below.

The following configuration types are loaded from `--add-dir` directories:

Configuration	Loaded from <code>--add-dir</code>
Skills in <code>.claude/skills/</code>	Yes, with live reload
Subagents in <code>.claude/agents/</code>	Yes
Settings in <code>.claude/settings.json</code> and <code>.claude/settings.local.json</code>	<code>enabledPlugins</code> and <code>extraKnownMarketplaces</code> keys only
CLAUDE.md files, <code>.claude/rules/</code> , and <code>CLAUDE.local.md</code>	Only when <code>CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1</code> is set. <code>CLAUDE.local.md</code> additionally requires the <code>local</code> setting source, which is enabled by default

Commands and output styles are discovered from the current working directory and its parents, your user directory at `~/.claude/` , and managed settings. Hooks and other `settings.json` keys load from the current working directory's `.claude/` folder with no parent-directory fallback, alongside your user `~/.claude/settings.json` and managed settings. To share that configuration across projects, use one of these approaches:

- **User-level configuration:** place files in `~/.claude/agents/` , `~/.claude/output-styles/` , or `~/.claude/settings.json` to make them available in every project
- **Plugins:** package and distribute configuration as a plugin that teams can install
- **Launch from the config directory:** run Claude Code from the directory containing the `.claude/` configuration you want

How permissions interact with sandboxing

Permissions and sandboxing are complementary security layers:

- **Permissions** control which tools Claude Code can use and which files or domains it can access. They apply to all tools, including Bash, Read, Edit, WebFetch, and MCP.
- **Sandboxing** provides OS-level enforcement that restricts the Bash tool's filesystem and network access. It applies only to Bash commands and their child processes.

Use both for defense-in-depth:

- Permission deny rules block Claude from even attempting to access restricted resources
- Sandbox restrictions prevent Bash commands from reaching resources outside defined boundaries, even if a prompt injection bypasses Claude’s decision-making
- Filesystem restrictions in the sandbox combine the `sandbox.filesystem` settings with Read and Edit deny rules; both are merged into the final sandbox boundary
- Network restrictions combine WebFetch permission rules with the sandbox’s `allowedDomains` and `deniedDomains` lists

When sandboxing is enabled with `autoAllowBashIfSandboxed: true`, which is the default, sandboxed Bash commands run without prompting even if your permissions include a bare `Bash` ask rule, or the **equivalent `Bash(*)` form**: the sandbox boundary substitutes for that whole-tool prompt. These checks still apply:

- Content-scoped ask rules like `Bash(git push *)` still force a prompt
- Explicit deny rules still apply
- `rm` or `rmdir` commands that target `/`, your home directory, or other critical system paths still trigger a prompt

Commands that won’t run sandboxed, such as excluded commands, respect the bare `Bash` ask rule as usual. See **sandbox modes** to change this behavior.

Managed settings

For organizations that need centralized control over Claude Code configuration, administrators can deploy managed settings that can’t be overridden by user or project settings. These policy settings follow the same format as regular settings files and can be delivered through MDM/OS-level policies, managed settings files, or **server-managed settings**. See **settings files** for delivery mechanisms and file locations.

Managed-only settings

The following settings are only read from managed settings. Placing them in user or project settings files has no effect.

Setting	Description
<code>allowAllClaudeAiMcps</code>	When <code>true</code> , claude.ai connectors load alongside a deployed <code>managed-mcp.json</code> instead of being suppressed by its exclusive control. See <u>Managed MCP</u>

Setting	Description
	<u>configuration</u>
<code>allowedChannelPlugins</code>	Allowlist of channel plugins that may push messages. Replaces the default Anthropic allowlist when set. Requires <code>channelsEnabled: true</code> . See <u>Restrict which channel plugins can run</u>
<code>allowManagedHooksOnly</code>	When <code>true</code> , only managed hooks, SDK hooks, and hooks from plugins force-enabled in managed settings <code>enabledPlugins</code> are loaded. User, project, and all other plugin hooks are blocked
<code>allowManagedMcpServersOnly</code>	When <code>true</code> , only <code>allowedMcpServers</code> from managed settings are respected. <code>deniedMcpServers</code> still merges from all sources. See <u>Managed MCP configuration</u>
<code>allowManagedPermissionRulesOnly</code>	When <code>true</code> , prevents user and project settings from defining <code>allow</code> , <code>ask</code> , or <code>deny</code> permission rules. Only rules in managed settings apply. Doesn't affect the MCP server allowlist; for that, set <code>allowManagedMcpServersOnly</code>
<code>blockedMarketplaces</code>	Blocklist of marketplace sources. Blocked sources are checked before downloading, so they never touch the filesystem. See <u>managed marketplace restrictions</u>
<code>channelsEnabled</code>	Allow <u>channels</u> for the organization. See <u>enterprise controls</u> for the default on each plan
<code>forceRemoteSettingsRefresh</code>	When <code>true</code> , blocks CLI startup until remote managed settings are freshly fetched and exits if the fetch fails. See <u>fail-closed enforcement</u>
<code>pluginTrustMessage</code>	Custom message appended to the plugin trust warning shown before installation
<code>sandbox.filesystem.allowManagedReadPathsOnly</code>	When <code>true</code> , only <code>filesystem.allowRead</code> paths from managed settings are respected. <code>denyRead</code> still merges from all sources
<code>sandbox.network.allowManagedDomainsOnly</code>	When <code>true</code> , only <code>allowedDomains</code> and <code>WebFetch(domain:...)</code> allow rules from managed settings are respected. Non-allowed domains are blocked automatically without prompting the user. Denied domains still merge from all sources
<code>strictKnownMarketplaces</code>	Controls which plugin marketplace sources users can add and install plugins from. See <u>managed marketplace restrictions</u>
<code>strictPluginOnlyCustomization</code>	Block skills, agents, hooks, and MCP servers from user and project sources, so they can only come from plugins or managed settings. <code>true</code> locks all four surfaces; an array such as <code>["skills", "hooks"]</code> locks only the named ones. See <u>strictPluginOnlyCustomization</u>
<code>wsInheritsWindowsSettings</code>	When <code>true</code> in the Windows HKLM registry key or <code>C:\Program Files\ClaudeCode\managed-settings.json</code> , WSL reads managed settings from the Windows policy chain in addition to <code>/etc/claude-code</code> . See <u>Settings files</u>

`disableBypassPermissionsMode` is typically placed in managed settings to enforce organizational policy, but it works from any scope. A user can set it in their own settings to lock themselves out of bypass mode.

❗ On Team and Enterprise plans, an Owner enables or disables [Remote Control](#) and [web sessions](#) organization-wide in [Claude Code admin settings](#). Remote Control can additionally be disabled per device with the `disableRemoteControl` setting. Web sessions have no per-device managed settings key.

Settings precedence

Permission rules follow the same [settings precedence](#) as all other Claude Code settings:

1. **Managed settings:** can't be overridden by any other level, including command line arguments
2. **Command line arguments:** temporary session overrides
3. **Local project settings** (`.claude/settings.local.json`)
4. **Shared project settings** (`.claude/settings.json`)
5. **User settings** (`~/.claude/settings.json`)

If a tool is denied at any level, no other level can allow it. For example, a managed settings deny can't be overridden by `--allowedTools`, and `--disallowedTools` can add restrictions beyond what managed settings define.

The same holds across settings scopes: if user settings allow a permission and project settings deny it, the deny rule blocks it. The reverse is also true: a user-level deny blocks a project-level allow, because deny rules from any scope are evaluated before allow rules.

Embedding hosts can supply additional managed policy via the SDK `managedSettings` option when `parentSettingsBehavior` is set to `"merge"`; embedder values can tighten policy but not loosen it.

Example configurations

This [repository](#) includes starter settings configurations for common deployment scenarios. Use these as starting points and adjust them to fit your needs.

See also

- [Settings](#): complete configuration reference including the permission settings table
- [Configure auto mode](#): tell the auto mode classifier which infrastructure your organization trusts
- [Sandboxing](#): OS-level filesystem and network isolation for Bash commands

- **Authentication**: set up user access to Claude Code
- **Security**: security safeguards and best practices
- **Hooks**: automate workflows and extend permission evaluation